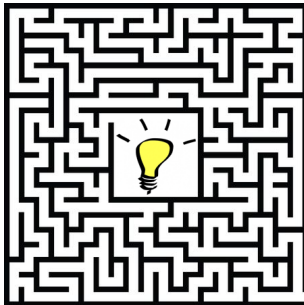


Advanced IT– Lösungsvorschläge

Prof. Dr. Henning Pagnia

DHBW Mannheim

Frühjahr 2026



Aufgabe 1 (man, cat)

- (a) Welche Aufgaben erfüllt der Befehl `cat`?
- (b) Erzeugen Sie mittels `cat` zunächst zwei kleine Dateien mit jeweils 4 Zeilen und anschließend eine dritte als Zusammenschluss der ersten beiden Dateien. Dabei sollen fortlaufende Zeilennummern vorangestellt werden!

Lösungsvorschlag

(a)

```
man cat
```

(b)

```
cat >file1  
cat >file2  
cat file1 file2 | cat -n >file3
```

(Drücken Sie jeweils Ctrl-D zum Eingabe-Ende)

Aufgabe 2 (echo, wc)

(a) Geben Sie unter Verwendung des Befehls `echo` den folgenden Text exakt wie angegeben auf dem Bildschirm aus:

```
Dear friend!
```

```
Welcome!
```

Tipp: 'man bash' und dort nach 'echo' suchen

(b) Überlegen Sie, welche Ausgabe die folgende Befehlszeile erzeugt:

```
echo >testf; echo >>testf; wc testf
```

Lösungsvorschlag

(a)

```
echo -e 'Dear friend!\n\n\n    Welcome!'
```

(b)

Es werden zwei Leerzeilen in `testf` geschrieben und anschließend die Anzahl an Bytes, Wörter und Zeilen der Datei ausgegeben.

Aufgabe 3 (sort)

Sortieren Sie die Zeilen einer Datei, so dass gleiche Zeilen in der Ausgabedatei nur ein einziges Mal vorkommen, wobei Groß- / Kleinschreibung ignoriert werden soll.

Beispieldatei 'Namen' zum Testen:

```
Mueller fehlt  
Maier fehlt  
Schmidt fehlt  
Maier fehlt  
Mueller fehlt  
Schmidt fehlt  
maier fehlt  
Schmidt fehlt  
Mueller fehlt  
schmidt      fehlt  
mueller FehLt
```

Die korrekte Ausgabe ist:

```
Maier fehlt  
Mueller fehlt  
schmidt      fehlt  
Schmidt fehlt
```

Lösungsvorschlag

```
sort -uf Namen
```

Aber nicht so:

```
sort -u Namen | sort -f
```

o. umgek.

Aufgabe 4 (uniq, head)

Finden Sie die am häufigsten auftretende Zeile einer Datei (ohne Berücksichtigung der Groß- / Kleinschreibung)!

Tipp: Verwenden Sie die Tools `sort`, `uniq` und `head`

Lösungsvorschlag

Die gewünschten Parameter sind '-ci' .

```
sort -f Namen | uniq -ci | sort -rn | head -n1
```

Zuerst wird die Datei sortiert (wichtig!), danach werden alle aufeinanderfolgenden identischen Zeilen gezählt, wobei die Groß- / Kleinschreibung ignoriert wird. Dann wird nochmal sortiert, so dass die häufigste Zeile ganz oben steht und schließlich nur diese ausgegeben.

Aufgabe 5 (Shell-Script)

(a) Schreiben Sie ein Shell-Script `mydel`, das eine angegebene Datei löscht, jedoch zuvor eine Sicherungskopie in Ihrem Verzeichnis `$HOME/Papierkorb/` anlegt.

Achten Sie darauf, dass auch sog. *Wildcards* ('*', '?') korrekt verarbeitet und auch Verzeichnisse gelöscht werden können!

Tipp: Sie können im Shell-Script die auf der Kommandozeile übergebenen Parameter referenzieren:

\$1 bezeichnet den ersten, \$2 den zweiten, ..., @\$ alle Parameter.

(b) Erstellen Sie in Ihrem Home-Verzeichnis ein Verzeichnis `bin/` und kopieren Sie Ihr Script dorthin in die Datei `rm`. Verändern Sie Ihren Datei-Suchpfad so, dass bei der Eingabe des Kommandos `rm` Ihre Datei aufgerufen wird. Wie können Sie nun den Papierkorb leeren?

Lösungsvorschlag

(a) Datei: mydel

```
#!/bin/bash  
echo Removing $@  
mv $@ $HOME/Papierkorb/
```

(b) Ausführbar machen, kopieren in das Verzeichnis, anpassen des Suchpfads

```
chmod 755 mydel  
cp mydel $HOME/bin/rm  
export PATH=$HOME/bin:$PATH
```

(c) Leeren des Papierkorbs geht dann mit

```
/bin/rm -r $HOME/Papierkorb/*
```

Aufgabe 6 (read, case)

Erstellen Sie ein Script, das solange von der Tastatur einen Namen einliest, bis ein 'q' oder ein 'quit' eingegeben wird. Für jeden Namen aus der nachstehenden Liste soll die passende Note angezeigt werden. Für alle anderen Namen soll '5.0' ausgegeben werden.

carla	1.0	uta	4.0
hans	1.0	georg	4.0
berta	4.0		

Lösungsvorschlag

```
while (true);
do
    echo -n "Geben Sie bitte den Namen ein: "
    read STUDENT
    case $STUDENT in
        q | quit ) exit;;
    esac
    echo -n "Die Note von $STUDENT ist "
    case $STUDENT in
        carla | hans ) echo "1.0";;
        uta | berta | georg ) echo "4.0";;
        * ) echo "5.0";;
    esac
done
```

Aufgabe 7 (awk)

Schreiben Sie ein `awk`-Programm, das zunächst vom Benutzer zwei Wörter einliest und dann in einer Eingabedatei alle Vorkommen des ersten Wortes durch das zweite ersetzt. Geben Sie das Ergebnis auf der Standardausgabe aus!

Tipp: Mit `getline str < "-"` können Sie eine Eingabe von der Tastatur in die Variable `str` einlesen.

Lösungsvorschlag

```
BEGIN { printf("From = ") > /dev/stderr";  
        getline FROM < "-"  
        printf("To = ") > /dev/stderr";  
        getline TO < "-"  
    }  
    { for(j=1; j<=NF; j++)  
        if ($j == FROM)  
            printf("%s ", TO);  
        else  
            printf("%s ", $j);  
        printf("\n");  
    }
```

Aufgabe 8 (Threads)

Bitte ändern Sie die Klasse *MyThread1* von Folie 30 so, dass der *main*-Thread unmittelbar nach Beendigung aller Threads die Nachricht "Alle fertig." auf dem Bildschirm ausgibt. Achten Sie dabei darauf, dass auch weiterhin alle Threads nebenläufig ablaufen!

Lösungsvorschlag

```
public class MyThread extends Thread {
    private int id;    // Thread ID
    public MyThread( int id ){
        this.id = id;
    } // Konstruktor

    public void run() {
        try {
            Thread.sleep( (int) ( Math.random() * 1000 ) );
        } catch (Exception e) {}
        System.out.println( "Hello World (ID = " + id + ")" );
    } // run

    public static void main( String[] args ) {
        Thread[] t = new Thread[9];
        for (int i=0; i<9; i++)
            t[i] = new MyThread( i ).start();
        try {
            for (int i=0; i<9; i++)
                t[i].join();
            System.out.println("Alle fertig!");
        }
        catch (Exception e) {System.out.println(e);}
    } // main
} //class MyThread
```

Die Referenzen auf die Threads müssen bei deren Erzeugung in einem Array oder in einer Array-List gespeichert werden. Nachdem alle Threads gestartet wurden, wird in einer weiteren Schleife mittels `join()` auf jeden einzelnen Thread gewartet. Wenn dann alle Threads beendet sind, wird die Nachricht auf dem Schirm ausgegeben.

Aufgabe 9 (Parallele Berechnungen)

(a) Schreiben Sie ein Java-Programm, das ein Array der Größe 2 097 152 ($= 2^{21}$) mit Integer-Werten > 0 füllt. Bilden Sie dann die Gesamtsumme und geben sie diese aus. Messen Sie die Zeit in Millisekunden, die das Programm nur für die Summenbildung benötigt hat.

(b) Wiederholen Sie das Vorgehen mit 2, 4, 8, 16, 32, 64, 128 und 256 Worker-Threads, die jeweils die Summe über einen eigenen Teil des Arrays bilden und an den Haupt-Thread zurückgeben und messen Sie ebenfalls jeweils die für die Summenbildung benötigte Zeit. Bauen Sie auch zusätzlich zur Summenbildung bei allen Threads zeitintensivere Berechnungen ein. Führen Sie verschiedene Messreihen durch!

Hinweis: Übergeben Sie eine fortlaufende ID an die Worker, so dass jeder Worker berechnen kann, welchen Ausschnitt des Arrays er bearbeiten muss.

(c) Diskutieren Sie die Ergebnisse!

Lösungsvorschlag

```
import java.util.Date;
```

```
public class SumWorker extends Thread {  
    static int MAX = 1; // Number of Worker Threads: 1,2,4,8, ...  
    static int anz = 1024 * 2048;  
    static int[] a = new int[anz];  
    static long[] erg = new long[MAX];  
    static SumWorker w[] = new SumWorker[MAX];  
    int myID = -1;  
  
    public SumWorker (int id){  
        myID = id;  
    }  
  
    public void run(){  
        double x;  
        int start = myID * (a.length / MAX);  
        int ende = (myID + 1) * (a.length / MAX);  
        long sum = 0;  
        for (int i = start; i < ende; i++){  
            sum = sum + a[i];  
            for (int j=1; j<20; j++) // sometime consuming computing  
                x = Math.log(Math.sqrt(3.1415 * (7^2) * j));  
        }  
    }  
}
```

```

    erg[myID]= sum;
}

public static void main( String[] args ) {
    // fill array
    for (int i=0; i<a.length; i++)
        a[i]= i;
    // create workers
    for (int i=0; i<MAX; i++)
        w[i] = new SumWorker(i);

    // start workers
    long date1 = System.currentTimeMillis();
    for (int i=0; i<MAX; i++)
        w[i].start();

    // wait for all workers to terminate
    try{
        for (int i=0; i<MAX; i++)
            w[i].join();
    } catch (InterruptedException e){System.err.println(e);}

    // compute result
    long sum = 0;
    for (int i=0; i<MAX; i++)
        sum += erg[i];
    long time = System.currentTimeMillis() - date1;
    System.out.println( "Used " + MAX + " Worker Threads - Result = " + sum +
        "      Computing Time = " + time );
}
}

```

Laufzeitmessungen (variieren, je nach System)

Threads	Min	Max
1	111 ms	355 ms
2	56 ms	73 ms
4	30 ms	43 ms
8	16 ms	27 ms
16	15 ms	19 ms
32	15 ms	20 ms
64	16 ms	19 ms
128	17 ms	22 ms
256	24 ms	31 ms

(Messwerte für 100 Messungen auf 2,4 GHz 8-Core Intel Core i9, 32 GB, macOS 13.5.1)

Der Aufwand für die Koordination der Threads (join ist ein System-Call) wirkt sich stärker aus je mehr Threads erzeugt werden. Bei zeitintensiveren Berechnungen, ggf. auch mit blockierenden E/A-Operationen, wirkt sich Multi-Threading i. Allg. stärker aus. Bei zu vielen Threads relativieren sich jedoch die Vorteile und die Zeiten steigen wieder an.

Aufgabe 10 (Verklemmungen)

Auf einem Betriebssystem, das Dijkstras Banker's Algorithm zur Vermeidung von Verklemmungen einsetzt, existieren die Betriebsmitteltypen **A**, **B**, **C**, **D** und die Prozesse **P₁**, **P₂**, **P₃**, **P₄**. Der aktuelle Zustand im System sei wie folgt:

Exist = (?, ?, ?, ?)

Avail = (1, 2, 3, 3)

max_needed(P_i):

	A	B	C	D
P₁ :	7	4	3	1
P₂ :	3	2	2	6
P₃ :	9	0	2	2
P₄ :	3	1	4	6

assigned(P_i):

	A	B	C	D
P₁ :	0	1	0	1
P₂ :	2	0	0	3
P₃ :	3	0	2	2
P₄ :	3	1	4	2

- Berechnen Sie den Betriebsmittelvektor **Exist** und die aktuelle max. Nachforderungsmatrix **needed(P_i)**.
- Wieso ist der resultierende Zustand unsicher, falls das Betriebssystem dem Prozess **P₃** eine weitere Instanz des Betriebsmittels **A** zuteilt?
- Ist der gegebene Zustand sicher? Falls ja, geben Sie eine der möglichen Ausführungsreihenfolgen und den resultierenden Verfügbarkeitsvektor an.

Lösungsvorschlag

(a) Betriebsmittelvektor **Exist** : (9, 4, 9, 11)

	A	B	C	D
P_1 :	7	3	3	0
needed (P_i) : P_2 :	1	2	2	3
P_3 :	6	0	0	0
P_4 :	0	0	0	4

(b) Unsicher, weil es keine mögliche Ausführungsreihenfolge für die weiteren Anforderungen gibt. Jeder Prozess könnte noch Betriebsmittel nachfordern wollen, bevor er sich beendet und seine Betriebsmittel freigeben würde.

(c) Ist sicher; die mögliche Ausführungsfolge ist $P_2 \Rightarrow P_4 \Rightarrow P_3 \Rightarrow P_1$.

$P_2 \Rightarrow \mathbf{Avail} = (3, 2, 3, 6)$

$P_4 \Rightarrow \mathbf{Avail} = (6, 3, 7, 8)$

$P_3 \Rightarrow \mathbf{Avail} = (9, 3, 9, 10)$

$P_1 \Rightarrow \mathbf{Avail} = (9, 4, 9, 11)$

Aufgabe 11 (Synchronisationsfehler)

Beim nebenläufigen Zugriff auf gemeinsam genutzte Datenstrukturen kann es zu Inkonsistenzen kommen. In dieser Aufgabe sollen Sie diese einmal am "lebenden System" beobachten.

- (a) Implementieren Sie in JAVA eine Klasse *FifoQueue* als eine einfach verkettete Liste, in der String-Objekte verwaltet werden können. Definieren Sie hierzu die beiden Methoden *put(String s)* sowie *String get()*.
- (b) Erzeugen Sie zwei Threads, die parallel auf der Queue arbeiten, und zeigen Sie, dass dabei durch die nebenläufigen Zugriffe von Zeit zu Zeit Fehler auftreten. (Hinweis: Verwenden Sie ggf. *Thread.yield()*-Aufrufe um eine Thread-Umschaltung zu forcieren.)
- (c) Sichern Sie die Methoden mittels eines Semaphors ab, und zeigen Sie, dass nun alles korrekt ausgeführt wird.

Lösungsvorschlag

```
import java.util.concurrent.Semaphore;
```

```
class FifoQueue {  
    public Elem head; // head element of list  
    boolean sync;     // use synchronization?  
    Semaphore sem;    // mutex Semaphore  
  
    public FifoQueue (boolean sync) {  
        head = null;  
        sem = new Semaphore(1, true);  
        this.sync = sync;  
    }  
  
    public void put (Elem newEl, int id) {  
        try {  
            if (sync) sem.acquire();  
            if ( head == null)  
                head = newEl;  
            else {  
                Elem el = head;  
                while (el.next != null) {  
                    el = el.next;  
                    // Thread.yield(); // if needed  
                }  
            }  
        }  
    }  
}
```

```

        el.next = newEl;
    }
    if (sync) sem.release();
} catch (InterruptedException e){e.printStackTrace();}
} // put

public String get (int id) {
    Elem el = null;
    try{
        if (sync) sem.acquire();
        el = head;
        if (el != null) {
            // Thread.yield(); // if needed
            head = el.next;
            el.next = null;
        }
        if (sync) sem.release();
    } catch (InterruptedException e){e.printStackTrace();}
    if (el == null) System.err.println("ERROR: get() bei leerer Queue!");
    return el.data;
} // get

public synchronized String toString() {
    String s = "";
    Elem el;
    for (el = this.head; el != null; el = el.next)
        s = s + "[" + el.data + "]" + " -> ";
    s += "NULL";
    return s;
} // toString

```

```
public void testIt (int id) {
    put(new Elem(id+":a"), id);
    put(new Elem(id+":b"), id);
    put(new Elem(id+":c"), id);
    get(id);
    get(id);
    put(new Elem(id+":d"), id);
    get(id);
    get(id);
} // testIt

public static void main (String[] args) {
    FifoQueue q;
    if (args.length == 1 && args[0].equals("sync")) {
        q = new FifoQueue(true);
        System.out.println("Using synchronization ...");
    }
    else {
        q = new FifoQueue(false);
        System.out.println("Running without synchronization ...");
    }
    FifoProcess pA = new FifoProcess(0, q);
    FifoProcess pB = new FifoProcess(1, q);
    pA.start();
    pB.start();
    try {
        Thread.sleep(700); // wait so that threads can terminate
    } catch (Exception e) {}
    System.out.println(q);
} // main
} // FifoQueue
```

```
class Elem {  
    public String data;  
    public Elem next = null;  
  
    public Elem (String data) {  
        this.data = data;  
    }  
} // Elem
```

```
class FifoProcess extends Thread {  
    int id;  
    FifoQueue q;  
    public FifoProcess (int id, FifoQueue q){  
        this.id = id;  
        this.q = q;  
    }  
    public void run () {  
        q.testIt(id);  
    } // run  
} // FifoProcess
```

Aufgabe 12 (5-Philosophen – Dijkstra 1971)

Fünf Philosophen sitzen gemeinsam an einem runden Tisch, an dem sie *unabhängig voneinander* von Zeit zu Zeit Spaghetti essen. Jeder Philosoph hat rechts neben seinem Teller nur eine Gabel, benötigt aber zum Essen zwei Gabeln, also auch die seines linken Nachbarn. Aus diesem Grund können nicht alle Philosophen gleichzeitig essen, sondern sie müssen sich bezüglich ihrer kritischen Abschnitte *Essen* synchronisieren.

- (a) Formulieren Sie die Synchronisationsbedingungen mittels eines Synchronisationsgraphen.
- (b) Geben Sie eine Java-Lösung unter Verwendung des Semaphorkonzepts an, wobei Sie die Philosophen jeweils als Threads implementieren sollten. Entwerfen Sie zunächst eine Lösung, bei der die Gabeln als Betriebsmittel aufgefasst werden.

(c) Entwerfen Sie nun eine zweite Lösung unter Verwendung des Konzepts der privaten Semaphore. Gehen Sie davon aus, dass ein Philosoph zyklisch die Zustände *denkend*, *hungrig*, *essend* durchlebt.

(d) Beurteilen Sie Ihre Lösungen jeweils anhand der Kriterien Verklemmungsfreiheit, Aushungerungsfreiheit und erreichbarer Parallelitätsgrad.

Lösungsvorschlag

(a) Synchronisationsgraph

Kritische Abschnitte sind „Phil. 1 isst“, „Phil. 2 isst“, usw.

Benachbarte kritische Abschnitte schließen sich jeweils gegenseitig aus.

(b) Die Gabeln als Betriebsmittel

Zur Vermeidung einer Verklemmung, Gabelbelegung immer in absteigender Reihenfolge:

`fork[4] vor fork[3] vor fork[2] vor fork[1] vor fork[0]`

D. h. vier Philosophen beantragen zuerst die *rechte* und dann die *linke* Gabel, der fünfte Philosoph umgekehrt.)

```
public class Philosoph1 {
    private Semaphore[] fork = new Semaphore[5];

    public Philosoph1 () {
        for ( int i=0 ; i<fork.length ; i++ )
            fork[i] = new Semaphore (1, true);
    }

    public void eat ( int i ) {
        // i ist der Index des Philosophen; muss zwischen 0 und 4 liegen
        if ( i==0 ) {
            fork[4].acquire();
            fork[0].acquire();
        }
        else {
            fork[i].acquire();
            fork[i-1].acquire();
        }
        // k.A.: essen;
        if ( i==0 ) {
            fork[4].release();
            fork[0].release();
        }
        else {
            fork[i].release();
            fork[i-1].release();
        }
    } // eat
} // Philosoph1
```

(c) Verwendung von privaten Semaphoren (lauffähige Lösung)

```
package AIT;
```

```
import java.util.concurrent.*;
```

```
public class PrivSemPhilos extends Thread {
```

```
    public static final int THINKING = 0;
```

```
    public static final int HUNGRY = 1;
```

```
    public static final int EATING = 2;
```

```
    int id = -1; // Philosophennummer
```

```
    // Nachfolgende Deklarationen muessen static sein!!!
```

```
    static Semaphore privsem[] = new Semaphore[5];
```

```
    static Semaphore mutex = new Semaphore(1,true);
```

```
    static private Semaphore semTrace = new Semaphore (1, true); // fuer korrekte Trace-Ausgaben
```

```
    static int state[] = new int[5];
```

```
    static PrivSemPhilos[] philo = new PrivSemPhilos[5];
```

```
    public PrivSemPhilos (int id){
```

```
        this.id = id;
```

```
        state[id] = THINKING;
```

```
        privsem[id] = new Semaphore(0,true);
```

```
    } // Konstruktor
```

```
    public void run() {
```

```
        for (int i=0; i<2; i++) {
```

```
            try {
```

```
                trace(id, "denkt");
```

```
                sleep(500+10*i); // denken simulieren
```

```
// Eintrittsprotokoll
trace(id, "will essen");
mutex.acquire();
if (state[(id+4) % 5] != EATING && state[(id+1) % 5] != EATING){
    state[id] = EATING;
    privsem[id].release();
}
else {
    state[id] = HUNGRY;
    trace(id, "muss warten");
}
mutex.release();
privsem[id].acquire(); // ggf. warten

trace(id, "isst");
sleep(500+20*id); // k.A.: essen simulieren
```

```
// Austrittsprotokoll
mutex.acquire();
trace(id, "ist satt");
state[id] = THINKING;
if (state[(id+4) % 5] == HUNGRY && state[(id+3) % 5] != EATING) {
    state[(id+4) % 5] = EATING;
    privsem[(id+4) % 5].release();
    trace(id, "weckt P-" + (id+4) % 5);
}
if (state[(id+1) % 5] == HUNGRY && state[(id+2) % 5] != EATING) {
    state[(id+1) % 5] = EATING;
    privsem[(id+1) % 5].release();
    trace(id, "weckt P-" + (id+1) % 5);
}
```

```

    }
    mutex.release();

} catch (Exception e) { e.printStackTrace(); }
} //for
} //run

public static void main(String[] args) throws Exception {
    for (int id=0; id<5; id++)
        philo[id] = new PrivSemPhilos(id);
    for (int id=0; id<5; id++)
        philo[id].start();           // starte Threads

    for (int id=0; id<5; id++)
        philo[id].join();
    System.out.println("*** ENDE - ALLE FERTIG ***");
} // main

private void trace(int id, String status) {
    try {
        semTrace.acquire();
        if (status.equals("denkt")){
            System.out.println ("P-" + id + " " + status);
        } else if (status.equals("will essen")){
            System.out.println ("\t\t P-" + id + " " + status);
        } else if (status.equals("muss warten")){
            System.out.println ("\t\t\t P-" + id + " " + status);
        } else if (status.equals("isst")){
            System.out.println ("\t\t\t\t P-" + id + " " + status);
        } else if (status.equals("ist satt")){
            System.out.println ("\t\t\t\t\t P-" + id + " " + status);
        }
    }
}

```

```
    } else if (status.startsWith("weckt")){  
        System.out.println ("\\t\\t\\t\\t\\t\\t\\t      P-"+id+" "+status);  
    }  
    semTrace.release();  
} catch(Exception e) {}  
} // trace  
  
} // class
```

(c) Elegantere Implementierungsvariante

```
public class Philosoph2 {
    private static final int THINK = 1;
    private static final int HUNGRY = 2;
    private static final int EAT = 3;
    private int[] state = new int[5];
    private Semaphore[] philos = new Semaphore[5];
    private Semaphore mutex = new Semaphore(1, true);

    public Philosoph2 () {
        for ( int i=0 ; i<5 ; i++ ) {
            state[i] = THINK;
            philos[i] = new Semaphore(0, true);
        }
    }

    private void test ( int i ) {
        if ( state[(i+4) % 5] != EAT && state[(i+1) % 5] != EAT && state[i] == HUNGRY ) {
            state[i] = EAT;
            philos[i].release();
        }
    }

    public void eat ( int i ) {
        mutex.acquire();
        state[i] = HUNGRY;
        test ( i );
        mutex.release();
        philos[i].acquire();
        // k.A.: essen;
    }
}
```

```
mutex.acquire();  
    state[i] = THINK;  
    test ( (i+4) % 5 );  
    test ( (i+1) % 5 );  
mutex.release();  
}  
} // Philosoph2
```

(d) Vergleich

Beide Lösungen sind verklemmungsfrei.

Lösung (b) ist aushungerungsfrei, garantiert aber nicht, dass zwei Philosophen gleichzeitig essen dürfen, wenn das möglich ist.

Lösung (c) garantiert max. Parallelität, ist aber nicht aushungerungsfrei. (Situation: beide Nachbarn könnten immer abwechselnd aber leicht überlappend essen.)

Aufgabe 13 (Monitorkonzept)

Lösen Sie das 5-Philosophenproblem mit dem Java-Monitorkonzept.

Tipp: Richten Sie für die Philosophen und den Monitor jeweils eine eigene Klasse ein. Achten Sie insbesondere darauf, dass alle Philosophen denselben Monitor verwenden!

Lösungsvorschlag

```
public class Philosoph3 extends Thread {  
    private int id;  
    private Monitor table;  
  
    public Philosoph3 (int id, Monitor table) {  
        this.id = id;  
        this.table = table;  
    } // Konstruktor  
  
    public void run () {  
        for (int k=0; k<8; k++) {  
            try {  
                Thread.sleep(1000 + id*200);  
            } catch (Exception e){}  
            table.startEating(id);  
            try {  
                Thread.sleep(1000 + id*200); // k. A. Essen  
            } catch (Exception e){}  
            table.stopEating(id);  
        }  
    } // run
```

```
public static void main ( String[] args ) {  
    int max = 5;  
    Monitor table = new Monitor(max);  
    Philosoph3 p[] = new Philosoph3[max];  
    for ( int id = 0 ; id < max ; id++ ) {  
        p[id] = new Philosoph3(id, table);  
        p[id].start();  
    }  
  
    for ( int id = 0 ; id < max ; id++ )  
        try { p[id].join(); } catch (Exception e){}  
    System.out.println("Fertig; alle satt :-)");  
} // main  
  
} // Philosoph3
```

```
public class Monitor {  
    private boolean[] eating;  
    private int max;  
  
    public Monitor (int max) {  
        this.max = max;  
        eating = new boolean[max];  
        for (int i=0; i<max; i++){  
            eating[i] = false;  
        }  
    } // Konstruktor  
  
    public synchronized void startEating ( int i ) {  
        while ( eating[(i+max-1) % max] || eating[(i+1) % max] ) {  
            System.out.println("Philosoph " + i + " wartet.");  
            try { wait(); } catch (Exception e){}  
        }  
        eating[i] = true;  
        System.out.println("Philosoph " + i + " isst.");  
    } // startEating  
  
    public synchronized void stopEating ( int i ) {  
        eating[i]= false;  
        System.out.println("Philosoph " + i + " denkt.");  
        notifyAll();  
    } // stopEating  
  
} // Monitor
```

Aufgabe 14 (TCP)

Entwerfen Sie mit TCP einen Server, der Nachrichten speichert und zur Abfrage über das Netz bereit hält.

Zum Ablegen einer Nachricht auf dem Server sendet ein Client einen String mit dem folgenden Format an den Server:

SAVE beliebig langer String mit abschließendem Zeilenende

Der Server generiert nach dem Empfang einen neuen geeigneten eindeutigen Schlüssel (als String) und speichert die Nachricht in einer Datei, wobei der Schlüssel als Dateiname verwendet wird. Danach sendet der Server den Schlüssel zurück an den Client:

KEY schluessel

Alle Dateien sollen auf dem Server auf dem Desktop im Verzeichnis "Messages/" abgespeichert werden. Achten Sie darauf, dass niemals Dateien überschrieben werden!

Zum Abrufen einer Nachricht sendet ein Client einen String:

GET schluessel

an den Server, der daraufhin überprüft, ob eine entsprechende Datei existiert. Falls ja, sendet er den Inhalt der Datei an den Client:

OK dateiinhalt

Anderenfalls sendet er:

FAILED

Implementieren Sie den Server als *Non-Persistent Server* auf Port 7777 sowie einen dazu passenden Client zum Testen.

Tipps:

- Verwenden Sie die Filter-Streams `PrintWriter` und `BufferedReader`.
- Verwenden Sie `split` aus der Klasse `String` zum Zerlegen der Nachrichten.
- Verwenden Sie die Klassen `FileReader` und `FileWriter` zum Zugriff auf die Dateien:
 `new BufferedReader(new FileReader(filename))`
 `new PrintWriter(new FileWriter(filename))`
- Erzeugen Sie einen wirklich eindeutigen Schlüssel für die Dateinamen.

Lösungsvorschlag

```
import java.net.*;
import java.io.*;

public class MessageClient {
    public static final int serverPort = 7777; // message server

    public static void main(String[] args) {
        String hostname = "localhost";
        if (args.length > 0) {
            hostname = args[0]; // get first command line parameter
        }

        // declare variables first
        PrintWriter networkOut = null;
        BufferedReader networkIn = null;
        Socket s = null;

        try {
            BufferedReader userIn = new BufferedReader(
                new InputStreamReader(System.in));
            while (true) {
                // close TCP connection after each command
                System.out.println("Type in message to send ('.' for exit) >");
                String theLine = userIn.readLine();
```

```
if (theLine.equals(".")) break;

s = new Socket(hostname, serverPort);
networkIn = new BufferedReader(
    new InputStreamReader(s.getInputStream()));
networkOut = new PrintWriter(s.getOutputStream());

networkOut.println(theLine);
networkOut.flush();
System.out.println(networkIn.readLine());

if (networkIn != null)
    networkIn.close();
if (networkOut != null)
    networkOut.close();
if (s != null)
    s.close();
} // end while

} // end try
catch (Exception e) {
    System.err.println(e);
}
} // end main

} // end MessageClient
```

```
import java.net.*;
import java.io.*;

public class MessageServer {
    public final static int DEFAULT_PORT = 7777;
    public final static String DIR = "Messages/";

    private static int uid = 5000;

    private ServerSocket server = null;

    public MessageServer(int port){
        try {
            server = new ServerSocket(port);
            System.out.println("Message Server successfully started on port " + port);
        } catch (IOException e) {
            System.err.println(e);
            System.exit(1);
        } // end catch
    } // Konstruktor

    public String genKey(){
        String key = "MF_" + uid;
        while (new File(DIR+key).exists()){
            uid++;
            key = "MF_" + uid;
        }
        return key;
    }
}
```

```
public void start(){
    Socket connection = null;
    PrintWriter out = null;
    BufferedReader in = null;
    String command = "";
    String result = "ERROR";
    while (true){
        try {
            connection = server.accept();
            in = new BufferedReader(new InputStreamReader(connection.getInputStream()));
            out = new PrintWriter(connection.getOutputStream());
            command = in.readLine();
            if (command != null){
                result = process(command);
                out.println(result);
            }
            out.close();
        } catch (IOException e) {System.err.println(e);}
        finally {
            try {
                // non-persistent connection: close connection after processing each command
                if (connection != null) connection.close();
            }
            catch (IOException e) {}
        }
    } // end while
} // end start
```

```
public String process(String command){
    String result = "ERROR";
    String key = "";
    String[] part = command.split(" ",2);
    if (part.length<2)
        return "FAILED: wrong command";
    if (command.startsWith("SAVE ")){
        try {
            key = genKey();
            PrintWriter f = new PrintWriter(new FileWriter(DIR + key));
            f.println(part[1]);
            result = "KEY " + key;
            f.close();
        } catch (Exception e){result = "FAILED: " + e;}
    }
    else if (command.startsWith("GET ")){
        key = part[1];
        try {
            BufferedReader br = new BufferedReader(new FileReader(DIR + key));
            result = br.readLine();
            if (result != null){result = "OK " + result;}
            br.close();
        } catch (Exception e){result = "FAILED: " + e;}
    }
    else // ambiguous command
        result = "FAILED: command unknown";
    return result;
} // process
```

```
public static void main(String[] args) {  
    int port = DEFAULT_PORT;  
  
    if (args.length > 0) {  
        try {  
            port = Integer.parseInt(args[0]);  
            if (port < 0 || port >= 65536) {  
                System.out.println("Port must between 0 and 65535");  
                return;  
            }  
        }  
        catch (NumberFormatException e) {}  
        // use default port  
    }  
  
    MessageServer messServ = new MessageServer(port);  
    messServ.start();  
  
} // main  
  
} // end MessageServer
```

Aufgabe 15 (UDP)

(a) Schreiben Sie ein einfaches UDP-basiertes Chat-Programm. Das Programm soll aus jeweils einem Client und einem Server bestehen. Jeder Client liest von der Tastatur eine Eingabezeile in Form eines Strings ein, der dann sofort zum Server gesendet wird. Der Server wartet auf Port 4999 und empfängt die Meldungen beliebiger Clients, die er dann – zusammen mit der IP-Adresse des Absenders – auf dem Bildschirm ausgibt.

(b) Starten Sie den Server auf Port 4998 und senden Sie Client-seitig *Broadcasts* in das lokale Netz.

Lösungsvorschlag

```
// CLASS ChatServer
```

```
import java.net.*;  
import java.io.*;
```

```
public class ChatServer{
```

```
    public final static int DEFAULT_PORT = 4999;  
    public final static int MAX_PACKET_SIZE = 65507;
```

```
    public static void main ( String[] args ) {  
        int port = DEFAULT_PORT;
```

```
        // determine serverport from command line
```

```
        if (args.length > 0){  
            try {  
                port = Integer.parseInt(args[0]);  
                if (port < 1 || port > 65535) throw new Exception();  
            }  
            catch (Exception e){  
                System.out.println("***** Usage: ChatServer portno");  
            }  
        } // if
```

```

// set up server
try {
    DatagramSocket socket = new DatagramSocket (port);

    System.out.println("*** ChatServer started at port " + port +
        ". Waiting for messages to display ...");

    byte[] buffer = new byte[MAX_PACKET_SIZE];
    DatagramPacket dp = new DatagramPacket ( buffer, buffer.length );

    while (true) {           // process messages as a daemon
        try {
            socket.receive(dp);           // wait for new input line

            String s = new String(dp.getData(),0, dp.getLength());

            System.out.println("* Message from " + dp.getAddress() +
                ":" + dp.getPort() + " <" + s + ">");
        } catch (Exception e){ System.err.println(e); }
    } // while
} // try
catch (Exception e) {
    System.err.println(e);
}
} // main
} // class

```

```
// CLASS ChatClient
```

```
import java.net.*;  
import java.io.*;
```

```
class ChatClient{  
    public final static int DEFAULT_PORT = 4999;  
    public final static int MAX_PACKET_SIZE = 65507;  
  
    public static void main ( String[] args ) {  
        BufferedReader userIn = new BufferedReader (  
                                new InputStreamReader(System.in));  
        String hostname = "localhost"; // Teil (b): "255.255.255.255"  
        int port = DEFAULT_PORT;  
        String s;  
        DatagramSocket socket = null;  
  
        // determine server name from command line arguments - if available  
        if (args.length > 1){  
            try {  
                hostname = args[0];  
                port = Integer.parseInt(args[1]);  
                if (port < 1 || port > 65535) throw new Exception();  
            }  
            catch (Exception e){  
                System.out.println("***** Usage: ChatClient hostname portno.");  
            }  
        }  
    }  
}
```

```

try {
    socket = new DatagramSocket ();
    InetAddress host = InetAddress.getByName(hostname);
    System.out.println("*** ChatClient: ready for sending packets " +
                        "to host " + host + " at port " + port + ".");
    System.out.println("Type messages to send! ('x' for exit)");

    do {
        System.out.print("> "); // print user prompt
        s = userIn.readLine(); // read user input line
        if ( s==null || s.equals("x")) break;

        byte[] data = s.getBytes(); // build packet
        int len = data.length;
        if (len > MAX_PACKET_SIZE)
            throw new Exception("Data too large to send");
        DatagramPacket dp = new DatagramPacket (data, len, host, port);
        socket.send (dp); // send packet
    } while ( true );
} // try
catch (Exception e) { System.err.println(e); }
finally {
    try {
        if (socket!= null) socket.close();
    } catch (Exception e) {}
}
} // main
} // class

```

Aufgabe 16 (File Server)

In dieser Aufgabe soll ein einfacher File-Server für Textdateien entwickelt werden. Vereinfachend gehen wir davon aus, dass dem Server ein festes Basisverzeichnis zugeordnet ist, in dem sich alle verwalteten Dateien befinden und dass er die notwendigen Zugriffsrechte besitzt. Die Textdateien sind dabei zeilenweise organisiert und beginnen mit Zeilennummer 1.

(a) Der Server soll auf Port 5999 Aufträge in Form von Strings mit
"READ filename,line_no"

entgegennehmen, wobei *line_no* eine positive ganze Zahl sein muss. Daraufhin wird vom Server die Datei *filename* geöffnet, die Zeile *line_no* ausgelesen und zurückgeschickt. Falls sich im Basisverzeichnis des Servers keine solche Datei befindet oder keine entsprechende Zeile vorhanden ist, soll an den Client eine Fehlermeldung zurückgesendet werden.

Implementieren Sie den Server sowie einen kleinen Test-Client (*keine Broadcasts senden!*). Verwenden Sie Java und UDP.

(b) Erweitern Sie den Server so, dass er auch das Kommando
 "WRITE filename,line_no,data"
versteht, bei dem die Zeile *line_no* durch *data* ersetzt werden soll.

Tipps:

(1) Implementieren Sie die Methoden *read* und *write* für die Verarbeitung der Dateizugriffe und lagern Sie diese in eine eigene Klasse aus (z. B. `class MyFile`)

(2) Verwenden Sie die folgenden Konstruktionen für die Dateizugriffe:

```
BufferedReader fileIn = new BufferedReader ( new FileReader(filename) );  
...  
String zeile = fileIn.readLine();
```

bzw.

```
PrintWriter fileOut = new PrintWriter ( new FileWriter(filename) );  
...  
fileOut.println(data);
```

Lösungsvorschlag

// Sequenzieller FileServer

```
import java.net.*;
import java.io.*;

public class FileServer {
    public static final int MAXSIZE = 65507;
    private static int port = 5999;
    private static String wdir = System.getProperty("user.home") + "/Desktop/FileServer/";

    public static void main(String[] args) {
        // declare variables first:
        String dpData = "";
        String answer = "*** ERROR 900: unknown error";
        String filename = "";
        String newData = "";
        int lineNo = -1;
        MyFile f = null;
        String param[] = null;
        String param2[] = null;
        DatagramSocket ds = null;
        DatagramPacket dp = null;
        DatagramPacket dp2 = null;
```

```
// set up server
try{ // 1
    ds = new DatagramSocket(port);
    System.out.println("Server successfully started on port "+port);

    while (true){ // wait for and process client requests
        try{ // 2
            dp = new DatagramPacket(new byte[MAXSIZE], MAXSIZE);
            ds.receive(dp);
            dpData = new String(dp.getData(),0,dp.getLength()).trim();

            if (dpData.startsWith("READ ")) {
                try{
                    param = dpData.split(" ",2);
                    param2 = param[1].split(",",2);
                    filename = param2[0].trim();
                    lineNo = Integer.parseInt(param2[1].trim());
                    f = new MyFile(wdir + filename);
                    answer = f.read(lineNo);
                } catch(Exception e){
                    answer = "*** ERROR 901: bad READ command";
                    throw new Exception(e);
                } //catch
            }

            else if (dpData.startsWith("WRITE ")){
                try{
                    param = dpData.split(" ",2);
                    param2 = param[1].split(",",3);
                    filename = param2[0].trim();
                    lineNo = Integer.parseInt(param2[1].trim());
                    newData = param2[2];
```

```
f = new MyFile(wdir + filename);
answer = f.write(lineNo, newData);
} catch (Exception e){
    answer = "*** ERROR 901: bad WRITE command";
    throw new Exception(e);
} //catch

} else {
    answer = "*** ERROR 902: unknown command";
    throw new Exception ("Unknown Command");
}

} catch (Exception e){ // try 2
    e.printStackTrace();
}

// send back result or error message
try{
    dp2 = new DatagramPacket(answer.getBytes(), answer.getBytes().length,
                             dp.getAddress(), dp.getPort() );

    ds.send(dp2);
} catch (Exception e){
    e.printStackTrace();
}
} //while

} catch (Exception e){ e.printStackTrace(); }
} //main
} //class
```

```
// CLASS MyFile
```

```
import java.io.*;
```

```
public class MyFile {
    String fileName;

    public MyFile (String fileName){
        this.fileName = fileName;
    }

    public String read (int lineNo){
        String answer = "*** ERROR 910: cannot open file for reading";
        BufferedReader f = null;
        try {
            f = new BufferedReader(new FileReader(fileName));
            String s = "*** ERROR 911: READ failed - line not found in file";
            for (int i = 0; (i<lineNo) && (s != null); i++)
                s = f.readLine();
            if (s != null)
                answer = s;
            else
                answer = "*** ERROR 911: READ failed - line not found in file";
        } catch (Exception e) {e.printStackTrace();}
        if (f != null){
            try{ f.close(); } catch(Exception e){e.printStackTrace();}
        }
        return answer;
    }
}
```

```
//read
```

```
public String write (int lineNo, String data){
    String answer = "*** ERROR 920: cannot open file for writing";
    BufferedReader inFile = null;
    PrintWriter outFile = null;
    boolean found = false; // is lineNo in inFile?

    try{
        inFile = new BufferedReader(new FileReader(fileName));
        outFile = new PrintWriter(new FileWriter(fileName+".temp"));
        answer = "*** ERROR 921: WRITE failed - line not found in file";
        String s = "";
        for (int i = 0; s != null; i++){
            s = inFile.readLine();
            if (i==lineNo-1){
                found = true;
                outFile.println(data);
            }
            else if (s!= null) {
                outFile.println(s);
            }
        } //for
    } catch(Exception e){ e.printStackTrace(); }

    if (inFile != null){
        try{ inFile.close(); } catch(Exception e){e.printStackTrace();}
    }

    if (outFile != null){
        try{ outFile.close(); } catch(Exception e){e.printStackTrace();}
    }
}
```

```
if (found){
    answer = data;
    try{
        // note that all files must be closed for renaming!
        File f1 = new File(fileName);
        File f2 = new File(fileName+".temp");
        File f3 = new File(fileName+".bak");
        f3.delete(); // possibly required for subsequent rename
        f1.renameTo(f3); // origin is new backup file
        f2.renameTo(f1); // make changes stable
    } catch(Exception e){e.printStackTrace();}
} //if

return answer;
} //write
} //class
```

```
// CLASS FileClient
```

```
import java.net.*;  
import java.io.*;
```

```
class FileClient{
```

```
    public final static int DEFAULT_PORT = 5999;  
    public final static int MAX_PACKET_SIZE = 65507;
```

```
    public static void main ( String[] args ) {  
        BufferedReader userIn = new BufferedReader (  
                                new InputStreamReader(System.in));  
        String hostname = "localhost";  
        int port = DEFAULT_PORT;  
        String s;
```

```
        // determine server from command line if available
```

```
        if (args.length > 1){  
            try {  
                hostname = args[0];  
                port = Integer.parseInt(args[1]);  
                if (port < 1 || port > 65535) throw new Exception();  
            }  
            catch (Exception e){  
                System.out.println("***** Usage: FileClient [hostname portno.]");  
            }  
        }  
    }
```

```

try {
    // create client datagram socket
    DatagramSocket socket = new DatagramSocket ();
    InetAddress host = InetAddress.getByName(hostname);
    System.out.print("Type command to send!");
    System.out.println("'READ file,lineNo' or 'WRITE file,lineNo,data' or 'x' to exit");

    do {
        System.out.print("> "); // print user prompt
        s = userIn.readLine(); // read user input line
        if ( s==null || s.equals("x")) break;
        byte[] data = s.getBytes(); // construct packet to send
        int len = data.length;
        if (len > MAX_PACKET_SIZE)
            throw new Exception("Data too large to send");
        DatagramPacket outPacket = new DatagramPacket (data, len, host, port);
        socket.send (outPacket); // send packet
        // create buffer for the server's answer
        byte[] buffer = new byte[MAX_PACKET_SIZE];
        DatagramPacket inPacket = new DatagramPacket ( buffer, buffer.length );
        socket.receive(inPacket);
        String answer = new String(inPacket.getData(),0, inPacket.getLength());
        System.out.println("** RECEIVED ANSWER: <" + answer + ">");
    } while ( true );
    socket.close();
} // try
catch (Exception e) {
    System.err.println(e);
}
} //main
} // class FileClient

```

Aufgabe 17 (Thread-per-Request)

In dieser Aufgabe soll unser bisheriger File-Server verbessert werden.

Modifizieren Sie Ihre bisherige Implementierung des File-Servers, sodass der Server den *Thread-per-Request* Ansatz (ohne Synchronisation) umsetzt!

Zeigen Sie durch ein geeignetes Szenario (und durch das Einfügen von sleep-Aufrufen an kritischen Stellen), dass der Server parallele Requests ausführt!

Was passiert mit Ihren Dateien bei parallelen Schreibzugriffen?

Lösungsvorschlag

```
// CLASS FileServer2
```

```
import java.net.*;
import java.io.*;

public class FileServer2 {
    public final static int DEFAULT_PORT = 4999;
    public final static int MAX_SIZE = 65507;

    public static void main ( String[] args ) {
        int port = DEFAULT_PORT;
        String msg = null;
        DatagramPacket inPacket = null;

        if (args.length > 0){
            try {
                port = Integer.parseInt(args[0]);
                if (port < 1 || port > 65535) throw new Exception();
            }
            catch (Exception e){
                System.err.println("***** Usage: FileServer [portno]");
            }
        } // if
    }
}
```

```

try { // depth 1
    DatagramSocket socket = new DatagramSocket (port);
    System.err.println("** FileServer started on port " + port + ". Waiting...");

    while ( true ) { // process messages as a daemon
        try { // depth 2
            // create fresh buffer for incoming
            inPacket = new DatagramPacket ( new byte[MAX_SIZE], MAX_SIZE );
            socket.receive(inPacket); // wait for incoming request

            // forward request to worker thread
            System.err.print(" *** Starting a worker thread ...");
            FileServer2Worker worker = new FileServer2Worker(inPacket, socket);
            Thread workerThread = new Thread(worker);
            workerThread.start(); // do not wait for thread but proceed
            System.err.println(" ok");
        } // try (2)
        catch (Exception e) {
            System.err.println("**** INTERNAL ERROR: " + e);
            System.err.println("SHUTTING DOWN SERVER");
            System.exit(1);
        }
    } //while
} // try (1)
catch (Exception e) {System.err.println(e);}
} // main
} // class FileServer2

```

```
// CLASS FileServer2Worker
```

```
import java.net.*;
import java.io.*;
```

```
class FileServer2Worker implements Runnable {
```

```
    DatagramPacket dp;
    DatagramSocket ds;
```

```
    public FileServer2Worker(DatagramPacket dp, DatagramSocket ds){
        this.dp = dp;
        this.ds = ds;
    }
```

```
    public void run() {
        String wdir = System.getProperty("user.home") + "/Desktop/FileServer/"; // base dir
        // declare variables first:
        String dpData = "";
        String answer = "*** ERROR 900: unknown error";
        String filename = "";
        String newData = "";
        int lineNo = -1;
        MyFile f = null;
        String param[] = null;
        String param2[] = null;
        DatagramPacket dp2 = null;
```

```
try{ // 1
    dpData = new String(dp.getData(),0,dp.getLength()).trim();

    if (dpData.startsWith("READ ")) {
        try{
            param = dpData.split(" ",2);
            param2 = param[1].split(",",2);
            filename = param2[0].trim();
            lineNo = Integer.parseInt(param2[1].trim());
            f = new MyFile(wdir + filename);
            answer = f.read(lineNo);
        } catch(Exception e){
            answer = "*** ERROR 901: bad READ command";
            throw new Exception(e);
        } //catch

    } else if (dpData.startsWith("WRITE ")){
        try{
            param = dpData.split(" ",2);
            param2 = param[1].split(",",3);
            filename = param2[0].trim();
            lineNo = Integer.parseInt(param2[1].trim());
            newData = param2[2];
            f = new MyFile(wdir + filename);
            answer = f.write(lineNo, newData);
        } catch(Exception e){
            answer = "*** ERROR 901: bad WRITE command";
            throw new Exception(e);
        } //catch
    }
}
```

```
    } else {  
        answer = "*** ERROR 902: unknown command";  
        throw new Exception ("Unknown Command");  
    } // if-else  
} catch (Exception e){ // try 1  
    e.printStackTrace();  
}  
  
// send back result or error message  
try{  
    dp2 = new DatagramPacket(answer.getBytes(), answer.length(),  
                               dp.getAddress(), dp.getPort() );  
    ds.send(dp2);  
} catch (Exception e){ e.printStackTrace(); }  
  
} // run  
  
} // class FileServer2Worker
```

Aufgabe 18 (Multi-Threaded Server Varianten)

Skizzieren Sie, wie der File-Server mit den folgenden Implementierungsvarianten aussehen würde:

(a) *Thread-per-File*

(b) *Worker-Pool*

Lösungsvorschlag

(a) Der Dispatcher verwaltet jeweils einen Worker-Thread pro Datei, welcher dann die Aufträge sequenziell abarbeitet. Da es auf den Dateien keine Parallelität gibt, benötigen wir keine Synchronisierung. Jedoch braucht jeder Worker eine eigene Auftragswarteschlange. Falls der Worker die Aufträge vom Dispatcher per (rechnerinternen) UDP Requests erhält, ist die Gesamtstruktur recht einfach und der Server später sogar leicht auf ein verteiltes System erweiterbar.

(b) Am besten verwendet man für die Übergabe der Aufträge vom Dispatcher an die bereits laufenden Worker-Threads einen Ringpuffer (E/V-Problem; s. Folien). Der Dispatcher erzeugt und startet die Worker-Threads jetzt gleich zu Beginn. Der Server fügt die neu angekommenen Aufträge in den Puffer ein (falls dort noch Platz ist) und wartet danach wieder auf den nächsten Auftrag. Auch die Worker laufen jetzt in einer Schleife und warten ggf. mittels *wait()* am Puffer, falls dieser leer ist. Sie holen sich einen Auftrag aus dem Puffer, bearbeiten ihn und beginnen dann wieder von vorne.

Vorsicht: Schachteln Sie keine synchronized-Methoden bzw. -Blöcke!